qss-odbc

ODBC databases from Emacs Lisp

A short manual

Friedrich Dominicus

Version 0.3.0

Contents

1 What it is **3**

2 Building the module **4**

3 Connecting and asking **5**
3.1 Connections 5
3.2 Queries and statements 5
3.3 Errors 5

4 Values **6**

5 Drivers **7**

6 Tests **8**

7 Files **9**

1 What it is

Any database with an ODBC driver — PostgreSQL, MariaDB, SQL Server, Access, SQLite … — from Emacs Lisp:

```
(require 'qss-odbc)
(setq con (qss-odbc-connect "test"))      ; a DSN, or a connection string
(qss-odbc-query con "SELECT id, name FROM person WHERE id > ?" 10)
;; => (("id" "name") (11 "Müller") (12 "Ærø"))
(qss-odbc-do-query ((id name) con "SELECT id, name FROM person")
  (message "%s %s" id name))
(qss-odbc-with-transaction con
  (qss-odbc-execute con "UPDATE person SET name = ? WHERE id = ?" "X" 11))
(qss-odbc-disconnect con)
```

Emacs Lisp has no foreign function interface, and Emacs will not get one: it would let any library into Emacs, free or not. What there is are dynamic modules, written in C against `emacs-module.h`. So the calls into ODBC live in a small module, `module/qss-odbc-module.c`, and `qss-odbc.el` is the library on top of it.

The division of labour: **C does what needs memory and pointers** — handles, buffers, UTF-16, binding parameters, reading columns piece by piece, collecting diagnostics — and returns raw values. **Lisp interprets them**: dates, exact decimals, booleans, the column descriptions. No pointer reaches Lisp, so no mistake in Lisp can crash Emacs.

The rules — which type is bound how, lengths in which unit, buffer sizes, the quirks of drivers — follow plain-odbc in its QSS version for Common Lisp, tested against Access, SQLite, PostgreSQL and MariaDB.

`qss` (`https://github.com/FDominicus/qss`) uses this package as a source of tables: `(qss-> (odbc "test" "person") (where (> :id 10)))`.

2 Building the module

```
M-x qss-odbc-build-module
```

It runs `make` in `module/`, shows the output in `*qss-odbc build*` and loads the result. It needs a C compiler, the ODBC headers and a driver manager:

Debian, Ubuntu	`apt install unixodbc-dev` (and a driver: `odbc-postgresql`, `odbc-mariadb`, `libsqliteodbc`)
macOS	unixODBC or iODBC; `ODBC=iodbc` for the latter
Windows	MinGW-w64 (MSYS2); the driver manager `odbc32` is built in

If `emacs-module.h` is not installed (an Emacs built in its source tree), set `qss-odbc-make-arguments` to `("EMACS_INCLUDE=/path/to/emacs/src")`. Emacs must be 29.1 or later and built with modules (`module-file-suffix` is not nil). If the module is missing, `qss-odbc-connect` offers to build it.

3 Connecting and asking

3.1 Connections

`(qss-odbc-connect DSN &rest OPTIONS)`: DSN is a data source name (`odbcinst -q -s`) or a full connection string. OPTIONS: `:user`, `:password`, `:autocommit` (default `t`), `:utf-8` (see chapter 5). The connection prints with the password hidden: `#<qss-odbc PostgreSQL DSN=test;UID=frido;PWD=***>`.

Connections start with autocommit on: every statement is its own transaction. `qss-odbc-with-transaction` runs a body in one, committed at the end and rolled back on any non-local exit; `qss-odbc-commit` and `qss-odbc-rollback` are there too.

3.2 Queries and statements

`qss-odbc-query`	rows as lists, the first the column names (as `sqlite-select` with `full`)
`qss-odbc-select`	`(COLUMNS . ROWS)`; columns as property lists `:name :type :size :scale :nullable :type-name`
`qss-odbc-select-all`	all result sets of a statement
`qss-odbc-execute`	a statement; the number of rows changed
`qss-odbc-map-query`	a function on each row, one row in memory at a time
`qss-odbc-do-query`	the same as a loop; `cl-return` leaves it
`qss-odbc-tables`, `-table-columns`, `-primary-key`	the catalog: SQLTables, SQLColumns, SQLPrimaryKeys; the last two take a schema

Values go in as parameters (the `?` in the SQL), never as text in the SQL. A statement the garbage collector finds unreferenced is freed; closing a connection closes its statements.

3.3 Errors

Errors are signalled as `qss-odbc-error` with the data `(MESSAGE SQLSTATE NATIVE-ERROR DIAGNOSTICS)`; DIAGNOSTICS holds every record. `qss-odbc-error-sqlstate` and `-message` take them apart:

```
(condition-case err (qss-odbc-query con "SELECT * FROM nothing")
  (qss-odbc-error (qss-odbc-error-sqlstate err)))   ; => "42P01"
```

4 Values

From the database	In Lisp
NULL	`nil`
integers	integers (bignums if need be)
REAL, FLOAT, DOUBLE	floats
NUMERIC(p,s), DECIMAL	integers scaled by 10^s: 1094.80 in a NUMERIC(10,2) is `109480`; `qss-odbc-decimal-string` shows it
BIT, BOOLEAN	`t`, `nil`
DATE, TIME, TIMESTAMP	text, the wall clock as stored: `"2026-10-30 09:56:00.123456"`; `qss-odbc-parse-time` makes a Lisp time
binary	unibyte strings
text, everything else	strings

Decimals are exact: a currency amount is never a float on the way. The scale is in the column (`qss-odbc-column-scale`), so 109480 and 2 are the amount; a column of a fixed scale — Access currency has 4 — adds and compares as integers.

Parameter	Bound as
`nil`, `t`	NULL, TRUE; FALSE is `(qss-odbc-bool nil)`
integer, float, string	INTEGER (beyond 32 bit NUMERIC), DOUBLE, WVARCHAR
`(qss-odbc-decimal 109480 2)`	NUMERIC 1094.80
`(qss-odbc-binary BYTES)`	VARBINARY
`(qss-odbc-date "2026-10-30")`	DATE
`(qss-odbc-timestamp "2026-10-30 09:56")`	TIMESTAMP (or a Lisp time, taken in UTC)

5 Drivers

Unicode drivers get UTF-16, through the W functions of ODBC. ANSI drivers that expect UTF-8 — SQLite, and psqlODBC's ANSI driver, often registered by mistake under the name *PostgreSQL Unicode* — get SQL text, parameters, character columns and messages as UTF-8 bytes. Through the W functions unixODBC would turn them into Latin-1 (SQLite lost even *ü*), and psqlODBC ANSI rejects UTF-16 parameters with HYC00 *Unrecognized C_parameter type*. Which drivers count as such: `qss-odbc-ansi-utf-8-drivers`, or `:utf-8 t` at `qss-odbc-connect`.

What the tests found:

SQLite reports VARCHAR through SQLColAttribute for BIGINT and BLOB columns (the type comes from SQLDescribeCol); keeps 15 digits of a double; has no exact numbers.

psqlODBC reports booleans as VARCHAR `"1"`/`"0"` (the type name `bool` decides); splits long messages into several records at the size of the first request.

MariaDB, MySQL have no boolean: BOOLEAN is TINYINT(1) and comes as 1 or 0. TEXT and BLOB end at 64 KB, TIMESTAMP covers 1970 to 2038 (LONGTEXT, LONGBLOB, DATETIME hold more). Connector/ODBC before 3.1.24 and 3.2.10 miscounts characters U+xxD8 to U+xxDB (∅). An UPDATE counts only rows whose values changed.

6 Tests

```
QSS_ODBC_DSNS="test mytest" emacs -Q --batch -L . -l qss-odbc-test \
  -f ert-run-tests-batch-and-exit
```

Unit tests run always; database tests against each DSN in `QSS_ODBC_DSNS` (default `test`), skipped with the reason if the module is not built or a DSN cannot be reached. They create and drop the table `qss_odbc_t`. Text in many scripts and with surrogate pairs on the piece boundaries of SQLGetData, integers at 32 and 64 bit, exact decimals, dates before 1900 and at both changes of summer time, binary data, errors, transactions, 2000 rows one at a time, 3000 statements left to the garbage collector, catalog functions.

Tested with PostgreSQL 16 (psqlODBC Unicode and ANSI, also a database that sorts by German rules), MariaDB 10.11 (Connector/ODBC 3.1.15) and SQLite3 ODBC; the module under valgrind without errors or leaks.

7 Files

`qss-odbc.el`	the library: connections, queries, values, the catalog, building the module
`module/qss-odbc-module.c`	the C module: handles, UTF-16, parameters, columns, diagnostics
`module/Makefile`	Linux, macOS (iODBC), Windows (MinGW)
`qss-odbc-test.el`	the tests
`doc/`	this manual